DES3UE WS 2026 Übung 4

Abgabetermin: siehe E-Learning, Abgabeform elektronisch

☒ **DES3UEG1 (Niklas)** **Name** Klemens Danner **Aufwand in h** 8
☐ **DES3UEG2 (Niklas)** **Punkte** _____ **Kurzzeichen Tutor*in** _____
☐ **DES3UEG3 (Teufl)**

Hinweise und Richtlinien:

- Übungsausarbeitungen müssen den im E-Learning angegebenen Formatierungsrichtlinien entsprechen – Nichtbeachtung dieser Formatierungsrichtlinien führt zu Punkteabzug.
- Zusätzlich zu den allgemeinen Formatierungsrichtlinien sind für diese Übungsausarbeitung folgende zusätzlichen Richtlinien zu beachten:
  - Treffen Sie, falls notwendig, sinnvolle Annahmen und dokumentieren Sie diese nachvollziehbar in ihrer Lösung!
  - Recherchieren Sie eventuell unbekannte Elemente nach Bedarf.

Ziel der Übung ist es, mithilfe der Sakila-Datenbank den Umgang mit PIVOT und UNPIVOT, analytischen Funktionen und dem LISTAGG-Operator zu üben. Eine zusätzliche Aufgabe behandelt im Selbststudium den Einsatz von künstlichen Schlüsseln.

Teilweise sind die Ergebnisse auszugsweise dargestellt. Geben Sie in Ihrer Lösung jeweils Screenshots ab, die größere Ausschnitte umfassen als in den Ergebnis-Auszügen dargestellt sind.

1. PIVOT und UNPIVOT **(6,5 Punkte** – 2 + 2 + 2,5**)**

1. Erstellen Sie eine Abfrage, die pro Sprache (L) und Original-Sprache (OL) die Anzahl an verfügbaren Filmen aus dem Jahr 2000 auflistet. Die Anzahl an Exemplaren im Inventar kann unbeachtet bleiben. Achten Sie darauf, dass es auch Filme ohne Original-Sprache geben kann. *(12 Zeilen)*

   Ergebnis-Auszug:

NUMFILME	SPRACHE	AR
6	Japanese	L
7	German	OL
6	English	OL
5	Mandarin	L

2. Erstellen Sie Tabelle, die für jede Kategorie (Zeilen) angibt, wie viele Verleihvorgänge an den einzelnen Wochentagen (Spalten: Montag bis Sonntag) stattgefunden haben. Ermitteln Sie den Wochentag aus dem Verleihdatum. Achten Sie darauf, dass Ihre Abfrage unabhängig von den Spracheinstellungen der Datenbank-Session funktioniert, indem Sie den passenden NLS-Parameter setzen. *(16 Zeilen, 8 Spalten)*

   Ergebnis-Auszug:

KATEGORIE	MONTAG	DIENSTAG	MITTWOCH	DONNERSTAG	FREITAG	SAMSTAG	SONNTAG
Action	159	160	154	167	150	160	162
Animation	166	168	159	158	171	178	166
Children	139	131	135	128	135	136	141
Classics	126	142	129	120	143	140	139

3. Sie sollen für **jede Kategorie** die **Anzahl der Filme pro Filiale** (Inventar) und **insgesamt** in der jeweiligen Kategorie berechnen. Erstellen Sie hierfür eine Kreuztabelle, die für Filialen und Kategorien die Anzahl der Filme (Inventar) berechnet, geben Sie auch die Gesamtsummen pro Filiale und pro Kategorie aus. Ergänzen Sie für die Erstellung der Kreuztabelle den gegebenen Code-Ausschnitt und verwenden Sie diese als Sub-Select eines Pivot-Befehls.

   Code-Ausschnitt (Angabe):

```
...
SELECT COUNT(*) AS anzahl,
       CASE WHEN /* ergänzen */ THEN 'Gesamt' ELSE name END AS Kategorie,
       CASE WHEN /* ergänzen */ THEN 'Gesamt' ELSE to_char(store_id) END AS Filiale
FROM inventory
    INNER JOIN film_category USING (film_id)
    INNER JOIN category USING (category_id)
GROUP BY /* ergänzen */ (store_id, name)
...
```

   Ergebnis-Auszug:

KATEGORIE	1	2	3	4	5	6	GESAMT
Action	48	47	46	72	55	44	312
Games	46	42	53	49	37	49	276
Drama	43	44	57	53	55	48	300
Comedy	43	47	45	41	50	43	269

2. Analytische Abfragen **(6 Punkte** – je 3**)**

Anmerkung: Verwenden Sie für die Lösung der folgenden Aufgaben analytische Funktionen!

1. Geben Sie zu jedem Film ID, Titel, Länge und die zugehörige Kategorie-Bezeichnung aus. Führen Sie außerdem pro Film an, wie viele **andere** Filme derselben Kategorie eine Länge haben, die **höchstens 5 Minuten** von der Länge des Films abweicht. (Wie viele Empfehlungen für "ähnliche" Filme gibt es?).
   Hinweis: Zur Selbstkontrolle sortieren Sie die Liste nach Kategorie und Länge. Für "Bride Intrigue" (Action) gibt es 11 weitere Filme, deren Laufzeit 5 Minuten kürzer oder länger ist.

   Ergebnis-Auszug:

	FILM_ID	TITLE	NAME	LENGTH	SUGGESTIONS
1	869	SUSPECTS QUILLS	Action	47	4
2	292	EXCITEMENT EVE	Action	51	6
3	111	CADDYSHACK JEDI	Action	52	6
4	542	LUST LOCK	Action	52	6
5	794	SIDE ARK	Action	52	6
6	697	PRIMARY GLASS	Action	53	8
7	97	BRIDE INTRIGUE	Action	56	11

2. Ermitteln Sie, welche Kund*innen schon einmal mehr als 180 Tage zwischen zwei Verleihvorgängen verstreichen haben lassen.

   a) Erstellen Sie zuerst eine analytische Abfrage, die für jede/n Kunde/in das aktuelle Verleihdatum und das nächste gegenüberstellt.

   b) Berechnen Sie aufbauend auf a) die Anzahl der Tage zwischen den Verleihvorgängen.

   Geben Sie für die geforderten Einträge den Vor- und Nachnamen der Kundin sowie die Anzahl der verstrichenen Tage aus.

Achtung: Die Auswertung ist nur möglich, wenn die Kundin bereits mehr als einen Film ausgeborgt hat (beachten Sie NULL-Werte).

Ergebnis-Auszug:

```
FIRST_NAME      LAST_NAME             TAGE
--------------- --------------- ----------
KATHLEEN        ADAMS                  238
KATHERINE       RIVERA                 185
EMILY           DIAZ                   190
ALICIA          MILLS                  190
```

3. LISTAGG **(8 Punkte** – 2 + 3 + 3**)**

a. Geben Sie zu jeder Kategorie eine Liste der neuesten Filme aus. Zusätzlich zum Titel des Films geben Sie die Laufzeit (Länge) in Klammern an. Sortieren Sie die Film-Liste so, dass die längsten Filme zuerst angeführt sind. Als „neueste Filme“ einer Kategorie gelten alle Filme mit dem jüngsten Erscheinungsjahr innerhalb dieser Kategorie.

Ergebnis-Auszug:

	NAME	FILME
1	Action	ANTITRUST TOMATOES (168), LORD ARIZONA (108), CLUELESS BUCKET (95)
2	Animation	WAIT CIDER (112)
3	Children	GRADUATE LORD (156), CROOKED FROGMEN (143)

b. Geben Sie für alle Filme, die 1991 erschienen sind, die Länge („length“) und den Filmtitel („film“) aus, sortieren Sie nach Länge absteigend.

Zusätzlich soll für jeden Film eine Liste aller Schauspieler*innen („actors“), die in dem Film mitspielen, ausgegeben werden (siehe Abbildung). In dieser Actors-Liste sollen die Namen nach Nachname und Vorname sortiert sein. Die Namen sollen so ausgegeben werden, dass jeweils der erste Buchstabe des Vornamens, ‘. ‘, und der Nachname angezeigt werden. Die einzelnen Schauspieler*innen sind durch Komma ‘,‘ voneinander zu trennen. Geben Sie auch Filme aus, in denen keine Schauspieler*innen mitspielen und verwenden Sie dafür den Text „no actors“. Für letzteres können Sie `REGEXP_REPLACE` oder `CASE … THEN … ELSE` verwenden.

Ergebnis-Auszug:

	LENGTH	FILM	ACTORS
1	184	CRYSTAL BREAKING	L. BERGMAN, P. CRONYN, R. KILMER, J. NEESON, F. WOOD
2	179	FLIGHT LIES	no actors
3	172	IDAHO LOVE	M. BOLGER, P. GOLDBERG, R. JOHANSSON, T. MCKELLEN, H. WILLIS, F. WOOD
4	171	JERICHO MULAN	V. BASINGER, W. HACKMAN, E. MARX, L. MONROE
5	167	ESCAPE METROPOLIS	N. HOPKINS, J. LOLLOBRIGIDA, E. MANSFIELD, F. TOMEI
6	164	VIRGINIAN PLUTO	J. BAILEY, K. BERRY, P. CRONYN, A. JOHANSSON, S. PECK, T. TEMPLE, G. WIL
7	163	QUEEN LUKE	R. DEAN, E. GOODING, S. PECK, J. SILVERSTONE, M. TANDY, R. WINSLET
8	161	OLEANDER CLUE	Z. CAGE, G. CHAPLIN, R. CLOSE, E. GOODING, W. JACKMAN, O. KILMER, S. KIL

c. Der Store in Linz (store_id = 3) möchte die aktuellen Interessen seiner Kund*innen näher bestimmen. Dafür sollen die Filmkategorien ermittelt werden, die die Kund*innen bei ihren **letzten drei Besuchen** ausgeliehen haben (als ein Besuch zählen Verleihvorgänge mit gleichem `rental_date`, berücksichtigen Sie also, dass pro Besuch mehrere Filme ausgeliehen werden können). Jede Kategorie soll nur einmal pro Kund*in aufgelistet werden. Die Liste kann daher je Person mehr oder weniger Kategorien enthalten.

Geben Sie Vor- und Nachname der Linzer Kundinnen als „customer“ und eine Liste der entsprechenden Kategorien (getrennt durch ‘,‘) als „recent_interests“ aus (siehe Ergebnis-Auszug).

Eine Vereinfachung können Sie unter anderem durch eine entsprechende Vorverarbeitung der Datenmenge (analytische Funktion) erreichen z.B. durch die Verwendung von WITH.

Fügen Sie ausreichend Testfälle in die Abgabe ein, um verschiedene Fälle zu demonstrieren (z.B. pro Kunde zwei Kategorien, drei Kategorien usw.).

Ergebnis-Auszug:

	CUSTOMER	RECENT_INTERESTS
1	GARCIA CAROL	Documentary, Classics, Animation
2	ALLEN SHIRLEY	Foreign, Children
3	HILL ANNA	Comedy, Drama
4	ADAMS KATHLEEN	Music, Children, Action
5	WATSON THERESA	Animation, Sports, Comedy
6	SANDERS TAMMY	Travel, Action, Drama
7	ALEXANDER DIANA	Games, Sports
8	DIAZ EMILY	Comedy, Animation, Family
9	ELLIS GRACE	Classics, Children, Games

4. Künstliche Schlüssel (Selbststudium) (3,5 Punkte)

Hinweis: Dieses Thema soll im Selbststudium erarbeitet werden. Arbeiten Sie das Skript *4_Künstliche_Schlüssel.sql* (E-Learning) Schritt für Schritt durch und lösen Sie anschließend untenstehendes Beispiel.

An einer Hochschule werden zwei Gruppen von Personen verwaltet: Student*innen und Mitarbeiter*innen. Die Daten werden aus Gründen der Zugriffstrennung in separaten Tabellen geführt (Basisrelationenmodell). Allerdings dürfen sich die IDs **nicht überschneiden,** damit jede Person an der Hochschule eindeutig identifiziert werden kann. Zudem soll über die ID sofort erkennbar sein, ob es sich um eine*n Student*in oder um eine/n Mitarbeiter/in handelt (unterschiedliche Nummernkreise).

Bilden Sie diese Anforderung nach, indem Sie die Tabellen „Student“ (PersonID, Vorname, Nachname, Semester) und „Mitarbeiter“ (PersonID, Vorname, Nachname, Gehalt) sowie weitere benötige Datenbankobjekte (Sequenzen) anlegen.

Fügen Sie beispielhaft je zwei Datensätze für Student*innen und Mitarbeiter*innen ein und überprüfen Sie anschließend mithilfe einer View (*Studenten_und_Mitarbeiter*), dass sich die vergebenen IDs nicht überschneiden und der Personentyp unmittelbar aus der ID hervorgeht. Dokumentieren Sie die Ergebnisse.

Aufgabe 1

1.1

Code

```
WITH tmp_table2 AS (
    SELECT film_id,
        language_id,
        original_language_id
    FROM film
    WHERE release_year = 2000
), unpiv_tab AS (
    SELECT *
    FROM tmp_table2
    UNPIVOT (
        lang_id
        FOR ar IN (
            language_id AS 'L',
            original_language_id AS 'OL'
        )
    )
)

SELECT
    COUNT(*) AS numfilme,
    name AS sprache,
    ar
FROM unpiv_tab
JOIN language ON lang_id = language.language_id
GROUP BY name, ar
ORDER BY name, ar;
```

Ausgabe

	NUMFILME	SPRACHE	AR
1	11	English	L
2	6	English	OL
3	5	French	L
4	6	French	OL
5	10	German	L
6	7	German	OL
7	6	Italian	L
8	3	Italian	OL
9	6	Japanese	L
10	5	Japanese	OL
11	5	Mandarin	L
12	6	Mandarin	OL

12 selected

1.2

Code

```
-- 1.2
WITH tmp_table AS (
    SELECT name AS category_name,
           rental_id,
           TO_CHAR(rental_date, 'FMDAY',
           'NLS_DATE_LANGUAGE=GERMAN') AS weekday
    FROM rental
        JOIN inventory USING (inventory_id)
        JOIN film_category USING (film_id)
        JOIN category USING (category_id)
)


SELECT *
FROM tmp_table
PIVOT (
    COUNT(rental_id)
    FOR weekday IN (
        'MONTAG' AS Montag,
        'DIENSTAG' AS Dienstag,
        'MITTWOCH' AS Mittwoch,
        'DONNERSTAG' AS Donnerstag,
        'FREITAG' AS Freitag,
        'SAMSTAG' AS Samstag,
        'SONNTAG' AS Sonntag
    )
);
```

Ausgabe

	CATEGORY_NAME	MONTAG	DIENSTAG	MITTWOCH	DONNERSTAG	FREITAG	SAMSTAG	SONNTAG
1	Documentary	142	166	163	159	132	143	145
2	Family	145	144	160	161	148	189	149
3	Children	139	131	135	128	135	136	141
4	Foreign	157	157	150	159	116	156	138
5	Classics	126	142	129	120	143	140	139
6	Sports	177	167	165	139	175	189	167
7	New	121	118	136	149	162	128	126
8	Travel	106	116	105	117	131	135	127
9	Animation	166	168	159	158	171	178	166
10	Horror	127	101	138	109	131	112	128
11	Action	159	160	154	167	150	160	162
12	Games	142	134	141	130	155	140	127
13	Comedy	157	138	139	120	135	127	125
14	Drama	176	140	153	144	147	150	150
15	Sci-Fi	152	140	166	156	150	167	170
16	Music	117	140	116	92	132	110	123

16 selected

1.3

Code

```
WITH tmp_table1 AS (
    SELECT COUNT(*) AS anzahl,
    CASE WHEN GROUPING(name) = 1 THEN 'Gesamt' ELSE name END AS Kategorie,
    CASE WHEN GROUPING(store_id) = 1 THEN 'Gesamt' ELSE to_char(store_id) END AS Filiale
    FROM inventory
    INNER JOIN film_category USING (film_id)
    INNER JOIN category USING (category_id)
    GROUP BY CUBE(store_id, name)
)

SELECT *
FROM tmp_table1
PIVOT (
    SUM(anzahl)
    FOR Filiale IN (
        '1' AS "1",
        '2' AS "2",
        '3' AS "3",
        '4' AS "4",
        '5' AS "5",
        '6' AS "6",
        'Gesamt' AS Gesamt
    )
);
```

Ausgabe

	KATEGORIE	1	2	3	4	5	6	GESAMT
1	Family	47	47	52	55	51	58	310
2	Foreign	53	45	53	49	49	51	300
3	Children	43	45	49	37	47	48	269
4	Classics	40	53	42	41	47	47	270
5	Documentary	50	52	50	55	46	41	294
6	New	52	45	43	46	43	46	275
7	Sports	51	61	54	60	72	46	344
8	Gesamt	757	755	753	780	784	752	4581
9	Horror	43	40	40	43	37	45	248
10	Travel	45	30	37	29	54	40	235
11	Animation	57	70	43	57	52	56	335
12	Action	48	47	46	72	55	44	312
13	Games	46	42	53	49	37	49	276
14	Drama	43	44	57	53	55	48	300
15	Comedy	43	47	45	41	50	43	269
16	Sci-Fi	52	52	57	54	47	50	312
17	Music	44	35	32	39	42	40	232

selected

Aufgabe 2

2.1

Code

```
-- 2.1
SELECT film_id, title, length, name,
       COUNT(*) OVER (PARTITION BY category_id ORDER BY length
                      RANGE BETWEEN 5 PRECEDING AND 5 FOLLOWING) - 1
                      AS suggestions
FROM film
    JOIN film_category USING (film_id)
    JOIN category USING (category_id)
ORDER BY category_id, length;
```

Ausgabe

FILM_ID	TITLE	LENGTH	NAME	SUGGESTIONS
869	SUSPECTS QUILLS	47	Action	4
292	EXCITEMENT EVE	51	Action	6
794	SIDE ARK	52	Action	6
542	LUST LOCK	52	Action	6
111	CADDYSHACK JEDI	52	Action	6
...				
1000	872	SWEET BROTHERHOOD	185	Travel	1

1000 selected

2.2

Code

```
--2.2 Welche KundInnen haben min 1 Mal mehr als 180 Tage
-- zwischen zwei Verleihvorgängen

-- a
WITH cmp_rental_dates AS (SELECT customer_id,
        rental_date,
        LEAD(rental_date) OVER
            (PARTITION BY customer_id ORDER BY rental_date)
                AS next_rental_date
FROM customer
    JOIN rental USING (customer_id)
)

-- b
, diff_rental_dates AS (SELECT customer_id,
        next_rental_date - rental_date AS diff_rental_date
FROM cmp_rental_dates
)

SELECT customer_id, first_name, last_name,
    MAX(diff_rental_date) AS max_diff_rental_date
FROM diff_rental_dates
    JOIN customer USING (customer_id)
GROUP BY customer_id, first_name, last_name
HAVING MAX(diff_rental_date) > 180;
```

Ausgabe

	CUSTOMER_ID	FIRST_NAME	LAST_NAME	MAX_DIFF_RENTAL_DATE
1	61	KATHERINE	RIVERA	185
2	99	EMILY	DIAZ	190
3	353	JONATHAN	SCARBOROUGH	280
4	486	GLEN	TALBERT	213
5	175	ANNETTE	OLSON	204
6	492	LESTER	KRAUS	181
7	505	RAFAEL	ABNEY	194
8	599	AUSTIN	CINTRON	226
9	152	ALICIA	MILLS	190
10	208	LUCY	WHEELER	193
11	36	KATHLEEN	ADAMS	238
12	318	BRIAN	WYMAN	191

12 selected

Aufgabe 3

3a

Code

```
WITH films_lastest_release AS
(SELECT name, category_id, title, length, release_year,
        MAX(release_year) OVER (PARTITION BY category_id)
            AS latest_release_year
FROM film
    JOIN film_category USING (film_id)
    JOIN category USING (category_id)
)

SELECT name,
        LISTAGG(title ||
            ' (' || length ||
            ')', ', ')
        WITHIN GROUP (ORDER BY length DESC) AS filme
FROM films_lastest_release
WHERE release_year = latest_release_year
GROUP BY category_id, name;
```

Ausgabe

	NAME	FILME
1	Action	ANTITRUST TOMATOES (168), LORD ARIZONA (108), CLUELESS BUCKET (95)
2	Animation	WAIT CIDER (112)
3	Children	GRADUATE LORD (156), CROOKED FROGMEN (143)
4	Classics	STEEL SANTA (143), DYNAMITE TARZAN (141), JEOPARDY ENCINO (102), SNATCHERS MONTEZUMA (74)
5	Comedy	SATURN NAME (182), HEAVEN FREEDOM (48)
6	Documentary	CHICKEN HELLFIGHTERS (122)
7	Drama	LUCK OPUS (152), LEBOWSKI SOLDIERS (69)
8	Family	GABLES METROPOLIS (161)
9	Foreign	MADNESS ATTACKS (178), EVERYONE CRAFT (163), DOUBTFIRE LABYRINTH (154), MATRIX SNOWMAN (56)
10	Games	VOLCANO TEXAS (157), SPY MILE (112)
11	Horror	AFFAIR PREJUDICE (117)
12	Music	YOUTH KICK (179), MASKED BUBBLE (151), JAWBREAKER BROOKLYN (118), REBEL AIRPORT (73)
13	New	FRONTIER CABIN (183), NUTS TIES (145), DRAGONFLY STRANGERS (133), VARSITY TRIP (85)
14	Sci-Fi	CITIZEN SHREK (165), CROWDS TELEMARK (112), SILVERADO GOLDFINGER (74)
15	Sports	NORTHWEST POLISH (172), ELIZABETH SHANE (152), GROOVE FICTION (111)
16	Travel	MADIGAN DORADO (116)

16 selected

3b

Code

```
SELECT length, title AS film,
        CASE WHEN COUNT(actor_id) = 0 THEN 'no actors'
        ELSE LISTAGG(SUBSTR(first_name, 1, 1)
             || '. '
             || last_name, ', ') -- sortierung der namen fertigstellen
            WITHIN GROUP (ORDER BY last_name, first_name)
            END AS actors
FROM film
    LEFT JOIN film_actor USING (film_id)
    LEFT JOIN actor USING (actor_id)
WHERE release_year = 1991
GROUP BY film_id, title, length
ORDER BY length DESC;
```

Ausgabe

	LENGTH	FILM	ACTORS
1	184	CRYSTAL BREAKING	L. BERGMAN, P. CRONYN, R. KILMER, J. NEESON, F. WOOD
2	179	FLIGHT LIES	no actors
3	172	IDAHO LOVE	M. BOLGER, P. GOLDBERG, R. JOHANSSON, T. MCKELLEN, H. WILLIS, F. WOOD
4	171	JERICHO MULAN	V. BASINGER, W. HACKMAN, E. MARX, L. MONROE
5	167	ESCAPE METROPOLIS	N. HOPKINS, J. LOLLOBRIGIDA, E. MANSFIELD, F. TOMEI
6	164	VIRGINIAN PLUTO	J. BAILEY, K. BERRY, P. CRONYN, A. JOHANSSON, S. PECK, T. TEMPLE, G. WILLIS, C. WRAY
7	163	QUEEN LUKE	R. DEAN, E. GOODING, S. PECK, J. SILVERSTONE, M. TANDY, R. WINSLET
8	161	OLEANDER CLUE	Z. CAGE, G. CHAPLIN, R. CLOSE, E. GOODING, W. JACKMAN, O. KILMER, S. KILMER, J. MOSTEL, P. PINKETT, S. WILLIAMS
9	155	GRAPES FURY	H. BERRY, L. DEE, C. HUNT, M. TANDY, B. TEMPLE
10	153	RIGHT CRANES	J. BAILEY, B. DUKAKIS, P. GOLDBERG, W. HURT, A. JOHANSSON, E. MARX, C. ZELLWEGER
11	152	TELEMARK HEARTBREAKERS	V. BASINGER, K. BERRY, M. BOLGER, S. GUINESS, G. HESTON, H. HOPE, J. MCQUEEN, J. NOLTE, G. SINATRA, S. SOBIESKI, H. VOIGHT
12	144	VIRTUAL SPOILERS	J. CHASE, G. PENN, D. TAUTOU
13	142	CHILL LUCK	M. HAWKE, W. JACKMAN, W. JOLIE, B. POSEY, M. TANDY
14	139	WORKER TARZAN	R. BACALL, Z. CAGE, S. DAVIS, J. DENCH, R. DUKAKIS, M. GIBSON, M. KILMER, J. NEESON, P. PINKETT
15	135	MONEY HAROLD	D. AKROYD, B. DUKAKIS, J. JACKMAN, P. MONROE, I. TANDY, A. WAYNE, G. WILLIS
16	133	MALLRATS UNITED	V. BASINGER, V. BOLGER, M. KEITEL, N. WAHLBERG, C. ZELLWEGER
17	130	AFRICAN EGG	M. CARREY, M. LEIGH, G. PHOENIX, D. TAUTOU, T. TEMPLE
18	129	GOSFORD DONNIE	H. GARLAND, A. HUDSON, J. RYDER

. . .

63	FORREST SONS	E. CHASE, R. CRAWFORD, M. KEITEL, W. TORN, D. WAHLBERG
59	COMMANDMENTS EXPRESS	C. BIRCH, B. DUKAKIS, W. JACKMAN, J. LOLLOBRIGIDA
51	CHAMPION FLATLINERS	J. CRUISE, F. DAY-LEWIS, G. WILLIS, C. WRAY

34 selected

3c

Code

```
WITH tmp_table AS (
    SELECT customer_id, first_name, last_name,
        inventory_id, rental_date,
        DENSE_RANK() OVER (
            PARTITION BY customer_id
            ORDER BY rental_date DESC
        ) AS visit_nr
    FROM customer
        JOIN rental USING (customer_id)
    WHERE customer.store_id = 3
), recent_categories AS (
    SELECT DISTINCT customer_id, first_name, last_name,
        name AS category_name
    FROM tmp_table
        JOIN inventory USING (inventory_id)
        JOIN film_category USING (film_id)
        JOIN category USING (category_id)
    WHERE visit_nr <= 3
)

SELECT last_name || ' ' || first_name AS customer,
        LISTAGG(category_name, ', ')
            WITHIN GROUP (ORDER BY category_name)
            AS recent_interests
FROM recent_categories
GROUP BY customer_id, first_name, last_name
ORDER BY last_name, first_name;
```

Ausgabe

	CUSTOMER	RECENT_INTERESTS
1	ADAMS KATHLEEN	Action, Children, Music
2	ALEXANDER DIANA	Games, Sports
3	ALLEN SHIRLEY	Children, Foreign
4	ARSENAULT KENT	Comedy, Drama, Foreign
5	ASHER TYRONE	Children, Classics, Documentary
6	BERRY REGINA	Documentary, Drama, Horror
7	BUFORD CLINTON	Children, Family, Foreign
8	BURKE LYDIA	Documentary, Games
9	CLARY ADRIAN	Classics, Family, Sports
10	COY GARY	Comedy, Foreign, Music
11	CULPEPPER EUGENE	Children, Documentary, Sports
12	CUNNINGHAM STACY	Documentary, Family, Horror
13	CURTIS LEAH	Children, Comedy, Travel
14	DIAZ EMILY	Animation, Comedy, Family
15	ELLIS GRACE	Children, Classics, Games
16	FOX HOLLY	Drama, Horror
17	GARCIA CAROL	Animation, Classics, Documentary
18	GILBERT TANYA	Animation, Family, Sci-Fi
19	HILL ANNA	Comedy, Drama
20	KRAUS LESTER	Documentary, Music, Travel

...

43	WAUGH JAMIE	Animation, Comedy, Family
44	WELCH MARLENE	Animation, Children

44 selected

leider keine Zeit mehr für Aufgabe 4, next time again :)